

CS409m: Introduction to Cryptography

Lecture 10 – Detailed Learning Notes

Prepared from the uploaded Lecture 10 slide deck

Course: CS409m, Instructor: Chethan Kamath

Lecture date in source: 02 September 2026

How to use these notes

This document is written as a teaching note rather than as a slide transcript. It keeps the terminology and main claims of the uploaded Lecture 10 slides, but expands the missing steps, gives explicit algorithms, and explains why the constructions work.

Important source-scope note: only one of the two files mentioned in the request was available in the current chat. Everything explicitly presented as “from the lecture” below is grounded in that uploaded Lecture 10 deck. Additional explanations are marked **Added by me**. The second file can be merged into this same document once it is provided.

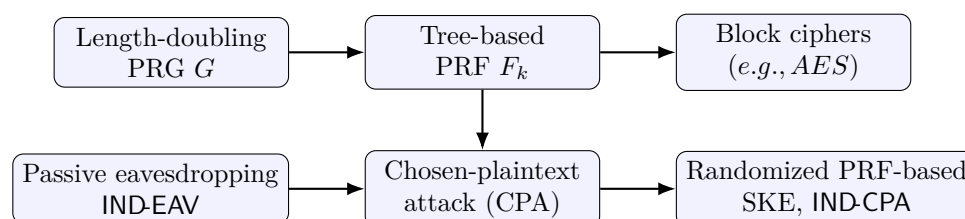
Contents

Lecture 10 Study Guide

2

Lecture 10 Study Guide

What this lecture is trying to teach The uploaded lecture has two main goals: (1) understand how to obtain a pseudorandom function (PRF) from a length-doubling pseudorandom generator (PRG), and (2) move from passive eavesdropping security to the stronger chosen-plaintext attack (CPA) model. The practical cryptographic object discussed alongside the theory is the block cipher, especially AES. The lecture then uses a randomized PRF-based encryption construction to obtain a stateless IND-CPA-secure symmetric-key encryption (SKE) scheme. The slides end by previewing block ciphers as pseudorandom permutations and modes of operation for long messages. These are the lecture’s stated topics and recap points.¹



Prerequisite notation – Added by me A few symbols make the lecture much easier to read:

- A bitstring $x \in \{0, 1\}^n$ has n bits. The empty string is written ε .
- $x||y$ means concatenate x and y .
- $x \oplus y$ is bitwise XOR, defined only for strings of the same length.
- A function is *negligible* if it becomes smaller than $1/p(n)$ for every polynomial p once n is sufficiently large. In cryptography, “negligible” means asymptotically too small to be a meaningful attack probability.
- A PPT adversary is a probabilistic polynomial-time algorithm: computationally feasible in the security parameter.

Recall from the previous lecture: PRF gives many pseudorandom one-time pads

The previous-lecture setup is a stateless symmetric-key encryption problem: the sender should be able to encrypt many messages without keeping evolving state, and the threat model is multi-message indistinguishability against an eavesdropper (IND-EAV). The lecture’s key observation is that a PRF lets us obtain pseudorandom pad material on demand. Instead of storing an exponentially large table of random pads, we store a short key k and evaluate $F_k(x)$ as needed.²

Added by me – extra learning material

Why the word “exponential” matters. If the input length is n , there are 2^n possible inputs to a PRF. In principle, the table of all values $F_k(x)$ contains 2^n pseudorandom-looking outputs. The PRF key is only n bits long (in the lecture’s tree-based construction), and one evaluation follows only a path of length n . Thus storage is compact while the

¹The slide deck explicitly lists the tree-based PRF, block ciphers/AES, CPA, randomized encryption, and the PRF-based stateless IND-CPA construction.

²The uploaded deck states that a PRF gives “efficient access to an exponential number of pseudorandom OTPs.”

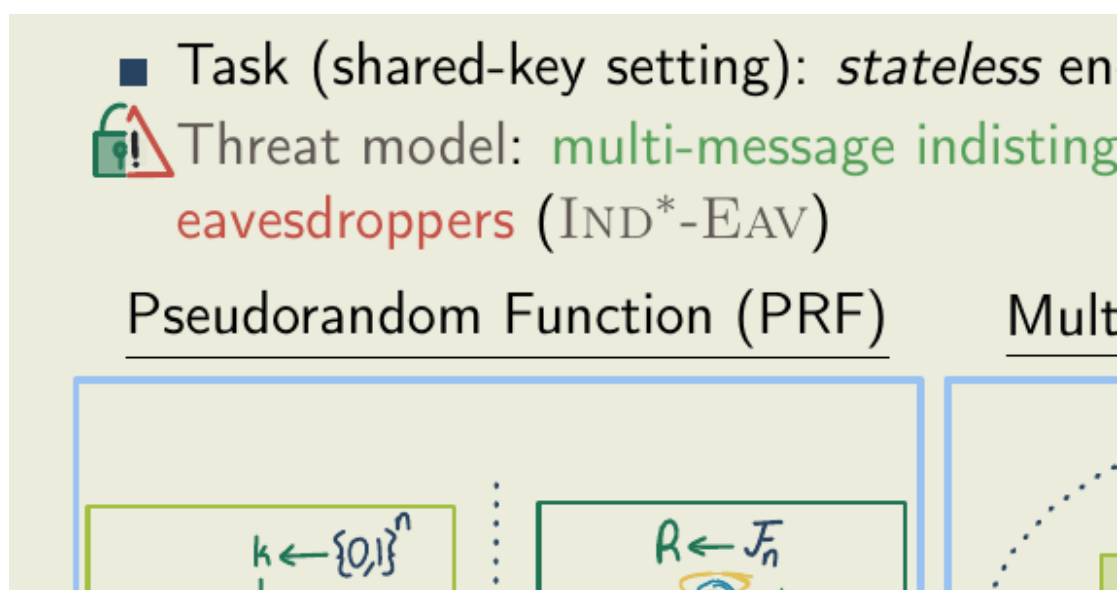


Figure 1: Source figure from the uploaded slide deck: PRF as an efficient way to access many pseudorandom one-time pads.

implicit table is exponentially large.

Topic 1: From a length-doubling PRG to a PRF The theoretical construction starts from a *length-doubling PRG*. Let

$$G : \{0,1\}^n \rightarrow \{0,1\}^{2n}$$

be a PRG. Write its output as two n -bit halves:

$$G(s) = G_0(s) \parallel G_1(s),$$

where

$$G_0, G_1 : \{0,1\}^n \rightarrow \{0,1\}^n.$$

The idea is to interpret $G_0(s)$ and $G_1(s)$ as the two pseudorandom “children” of a node containing s . Repeating this for n levels gives a binary tree of depth n with 2^n leaves.

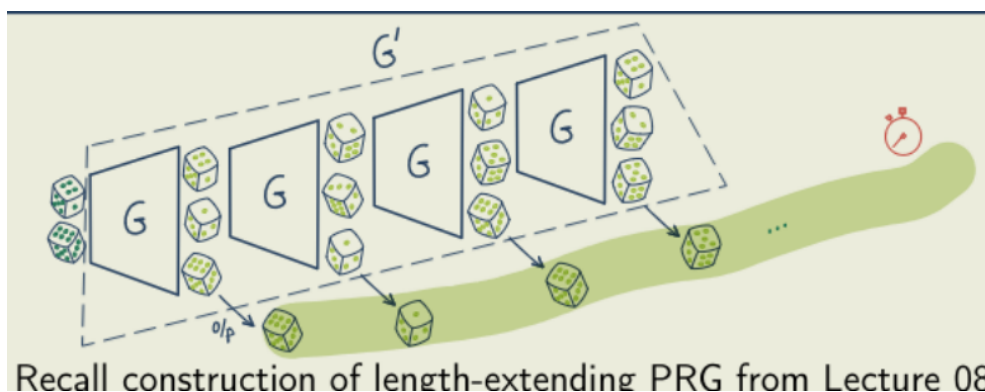


Figure 2: Source figure from the lecture: direct expansion creates exponentially many nodes; the PRF construction will access only one path.

Intuition

Imagine a tree whose root stores the secret key. Each application of G doubles the number of available states. At depth 1 there are two states; at depth 2 there are four; after n levels there are 2^n leaves. A naïve algorithm that generated the whole tree would take exponential time. The trick is that, for one input x , we only need the single root-to-leaf path selected by the bits of x .

Tree-based PRF construction The construction in the slides defines a keyed function family

$$\{F_k : \{0,1\}^n \rightarrow \{0,1\}^n\}_{k \in \{0,1\}^n}.$$

Given key k and input

$$x = x_1x_2 \cdots x_n \in \{0,1\}^n,$$

start at the root with

$$s_\varepsilon := k.$$

Then, for each input bit x_i , follow the corresponding PRG branch:

$$s_i := G_{x_i}(s_{i-1}), \quad i = 1, \dots, n.$$

Finally output

$$F_k(x) := s_n.$$

This is exactly the meaning of the slide's notation " $F_k(x) = s_x$ with $s_\varepsilon := k$ ": the subscript x names the node reached by following the bits of x from the root.

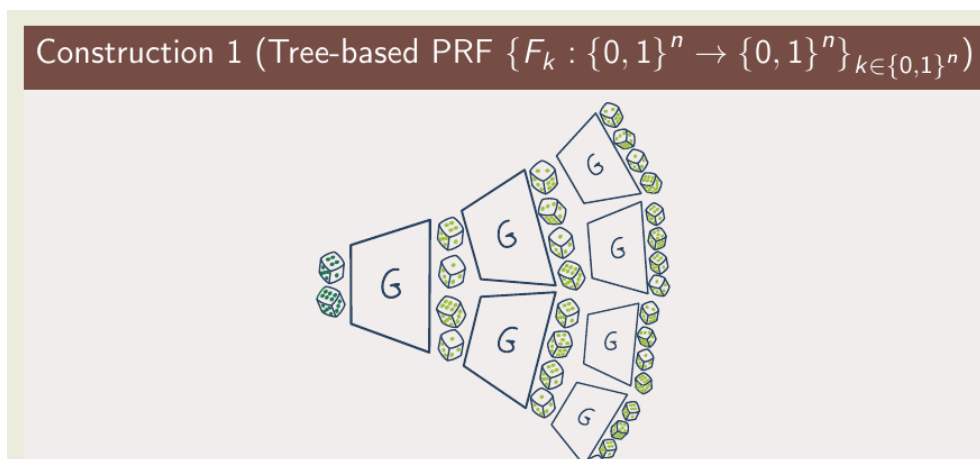


Figure 3: Source figure from the lecture showing the binary tree induced by repeated applications of G .

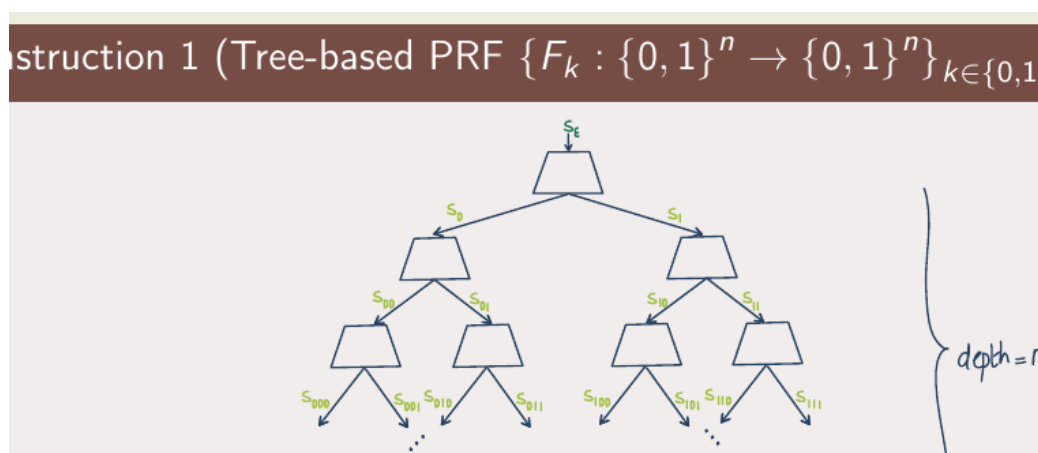


Figure 4: Source figure from the lecture emphasizing the depth- n tree and the definition of $F_k(x)$.

Tree-based PRF pseudocode

```

F(k, x[1..n]):
  s ← k
  for i ← 1 to n:
    (s0, s1) ← G(s)           # split 2n bits into two n-bit halves
    if x[i] = 0:
      s ← s0
    else:
      s ← s1
  return s

```

Equivalently, with $G(s) = G_0(s) \parallel G_1(s)$, the update is $s \leftarrow G_{x_i}(s)$.

Pseudocode

Added by me – extra learning material

A tiny worked example. Take a toy input length $n = 3$ and input $x = 101$. Starting from $s_\epsilon = k$, we compute

$$s_1 = G_1(k), \quad s_2 = G_0(s_1), \quad s_3 = G_1(s_2),$$

so $F_k(101) = s_3$. We never generate the other five leaves at depth 3.

Why the construction is efficient There are 2^n leaves in the full tree, but evaluating one output requires only n PRG evaluations. Thus, assuming one PRG evaluation is efficient, one PRF evaluation is also efficient. This is the key “locality” property emphasized by the lecture: the function has an exponentially large conceptual output table, while an individual query touches only a path of length n .

Why the construction is a PRF The lecture states the central theorem:

Theorem / Key Claim

If G is a length-doubling PRG, then the tree-based construction is a PRF.

The slides point to an advanced hybrid argument for the formal proof. The core proof idea can be understood without reproducing the full formal hybrid bookkeeping.

Added by me – extra learning material

Proof intuition for the theorem. Replace the pseudorandom outputs of the PRG by truly random child labels, one conceptual tree layer/region at a time. If G is a secure PRG, an efficient distinguisher should not notice the replacement except with negligible advantage. Once every branch is replaced by independent random labels, each input x lands on a leaf that is random and independent from the other leaves, so the resulting function behaves like a truly random function on $\{0, 1\}^n$. The hybrid argument sums the small distinguishing gaps over polynomially many hybrids, preserving negligibility. The full proof is more technical because the tree contains exponentially many potential nodes; the standard GGM proof handles this carefully.

A critical subtlety: which side of the tree uses the key? The construction starts with the key at the root and uses the input bits to select branches. Thus

$$F_k(x) : \quad \text{key fixed, input chooses the path.}$$

The slide asks what happens if we instead set $s_\varepsilon := x$ and use s_k as the output. That reverses the roles of key and input. It is important because a PRF is a keyed family in which the *key selects the function* while the *input is the public query*. Swapping them does not preserve the same security interpretation: the object must remain efficiently evaluable for arbitrary public inputs without exposing the secret key.

Common Pitfall

For exams, do not confuse the following two ideas:

- k is secret and identifies one member F_k of the family.
- x is the input to that fixed member.

The notation $F_k(x)$ should be read as “evaluate the function chosen by secret key k at public input x .”

Theoretical PRFs from hard problems The lecture gives the following corollary: PRFs exist if certain underlying computational problems are hard, including integer factoring (more precisely, the slide mentions quadratic residuosity) and subset sum. The stated logic is:

$$\text{hard problem} \Rightarrow \text{secure PRG} \Rightarrow \text{tree-based PRF.}$$

The exact reductions to the relevant PRGs belong to earlier lectures/referenced material; this Lecture 10 deck cites Goldreich’s textbook for the PRF proof.

Topic 2: Block ciphers in practice In practice, the lecture points to *block ciphers*, with AES (Advanced Encryption Standard) as the central example. The slide emphasizes that AES has fixed key sizes of 128, 192, and 256 bits. A block cipher is not a drop-in “encrypt an arbitrarily long file with one formula” primitive; for messages longer than one block, a *mode of operation*

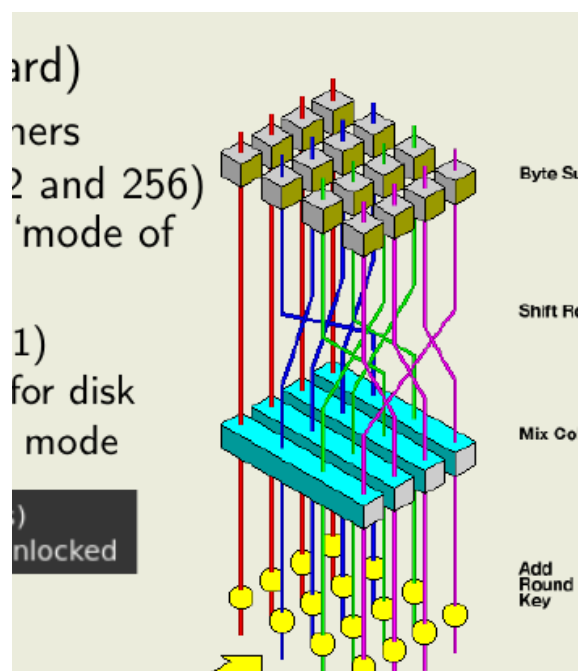


Figure 5: AES structure figure included in the lecture slides.

specifies how the block cipher is used across many blocks. The lecture lists GCM and XTS as examples and notes that LUKS disk encryption uses AES in XTS mode.

Added by me – extra learning material

PRF versus block cipher. A PRF is modeled as a keyed function that should be indistinguishable from a random function. A block cipher is modeled as a keyed permutation on fixed-length blocks. The next lecture explicitly previews the block cipher / pseudorandom permutation (PRP) viewpoint. The distinction matters: a permutation is bijective, while a random function need not be bijective.

AES engineering facts from the lecture The uploaded deck highlights three practical reasons block ciphers are ubiquitous: they are built into major cryptographic libraries, they are widely deployed, and modern processors provide dedicated acceleration such as Intel AES-NI. These are implementation facts rather than security definitions, so keep them conceptually separate from the PRF security theorem.

Topic 3: Chosen-Plaintext Attack (CPA) The lecture now strengthens the threat model. Under passive eavesdropping, the adversary only observes ciphertexts. Under a chosen-plaintext attack, the adversary can interact with an encryption oracle and adaptively request encryptions of messages of its choice. The reason this is realistic is that attackers can sometimes cause a system to encrypt attacker-chosen data even without knowing the secret key.

Intuition

The key difference is not “the adversary knows plaintexts” versus “does not know plaintexts.” In many real protocols, plaintexts are already predictable. The stronger CPA model gives the adversary explicit control over encryption queries, which lets it test how the encryption scheme responds to carefully chosen inputs.

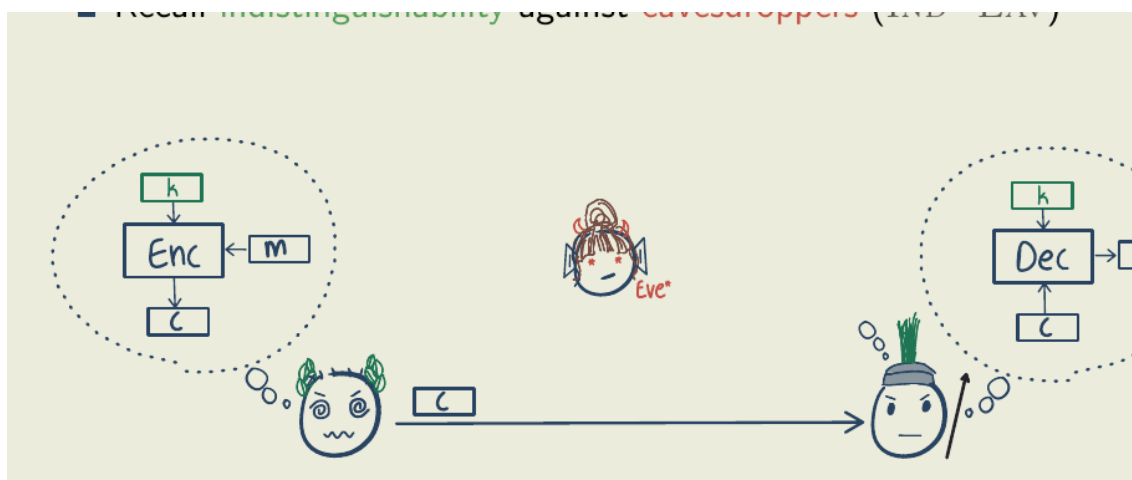


Figure 6: Passive eavesdropping picture from the lecture.

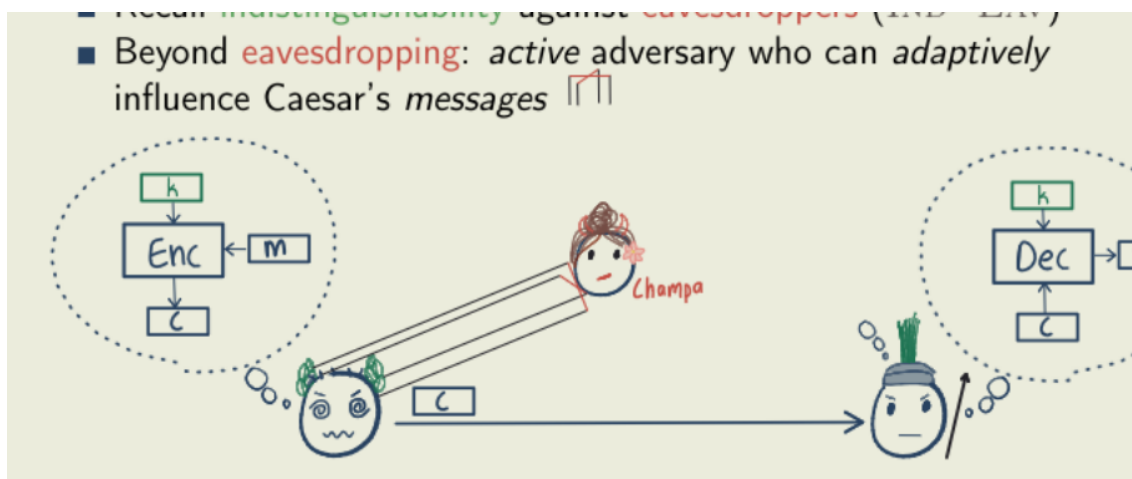


Figure 7: CPA picture from the lecture: the adversary can influence which plaintext gets encrypted.

IND-CPA game – the version to remember Let $\Sigma = (\text{Gen}, \text{Enc}, \text{Dec})$ be a symmetric-key encryption scheme. A standard IND-CPA experiment can be written as follows.

1. The challenger samples $k \leftarrow \text{Gen}(1^n)$.
2. The adversary gets access to an encryption oracle $\text{Enc}_k(\cdot)$ and may make adaptive queries.
3. The adversary chooses two equal-length challenge messages m_0, m_1 .
4. The challenger samples a random challenge bit $b \leftarrow \{0, 1\}$ and returns

$$c^* \leftarrow \text{Enc}_k(m_b; r^*)$$

where r^* represents fresh encryption randomness when the scheme is randomized.

5. The adversary continues making allowed oracle queries and outputs b' .

The scheme is IND-CPA-secure when every PPT adversary has only negligible advantage over random guessing:

$$\Pr[b' = b] \leq \frac{1}{2} + \text{negl}(n).$$

Equivalently, the advantage is

$$\text{Adv}_{\Sigma, A}^{\text{ind-cpa}}(n) := \left| \Pr[b' = b] - \frac{1}{2} \right|,$$

and this quantity must be negligible for every PPT adversary A .

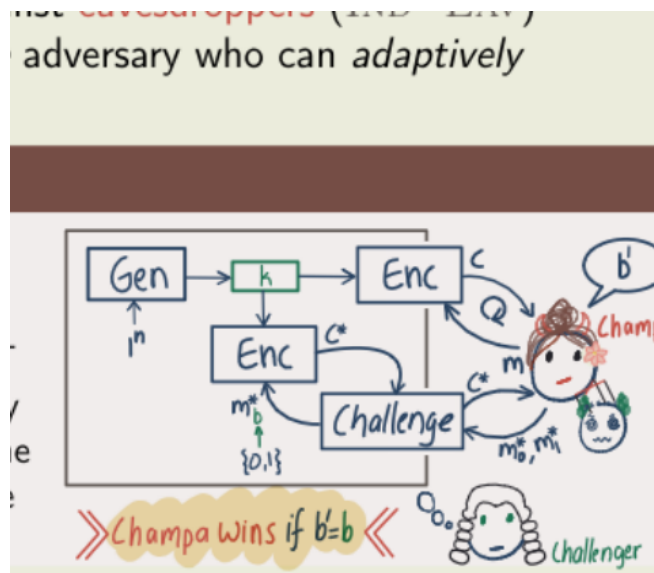


Figure 8: Challenge-game picture from the lecture.

Why randomized encryption is necessary The lecture asks whether an IND-CPA-secure SKE can have deterministic encryption. The answer is no. Suppose there is a deterministic encryption algorithm $c = \text{Enc}_k(m)$. Then the adversary can exploit equality preservation.

For a challenge pair (m_0, m_1) , ask the encryption oracle for $\text{Enc}_k(m_0)$ before the challenge and obtain c_0 . When the challenge ciphertext c^* arrives, compare it with c_0 :

$$c^* = c_0 \Rightarrow b = 0 \quad (\text{for the chosen pair}).$$

If $c^* \neq c_0$, the adversary can infer that the other challenge message was used (assuming the chosen messages are distinct and have distinct deterministic ciphertexts, which correctness of the scheme requires for this simple attack).

The important lesson is broader than the particular test: deterministic encryption leaks whether two plaintexts are equal. IND-CPA requires that an adversary cannot use such information to distinguish the challenge bit.

Added by me – extra learning material

Mental model: “fresh randomness breaks equality tests.” If the same message is encrypted twice, randomized encryption should normally produce unrelated-looking ciphertexts (except for metadata that the particular scheme intentionally reveals). That is the intuition behind why the challenge ciphertext cannot simply be compared with a previous oracle answer.

Real-world CPA example mentioned in the lecture The lecture gives a historical example from World War II in which British mine locations were used to induce encrypted

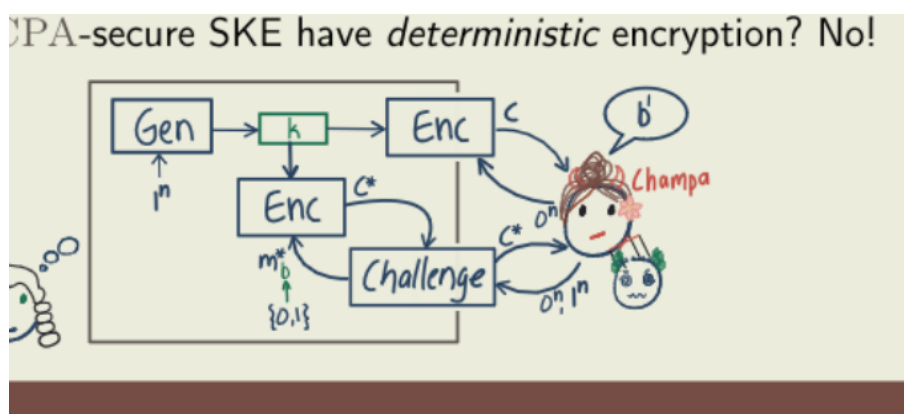


Figure 9: Deterministic-encryption attack shown in the lecture.

reports, illustrating the general idea of an attacker influencing the plaintexts that get encrypted. It also notes that computer viruses may be able to cause a machine to encrypt attacker-controlled messages without directly obtaining the secret key.

During World War II, British placed mines at certain locations, knowing that the Germans—when finding those mines—would report the locations and send them back to headquarters.” [KL14]

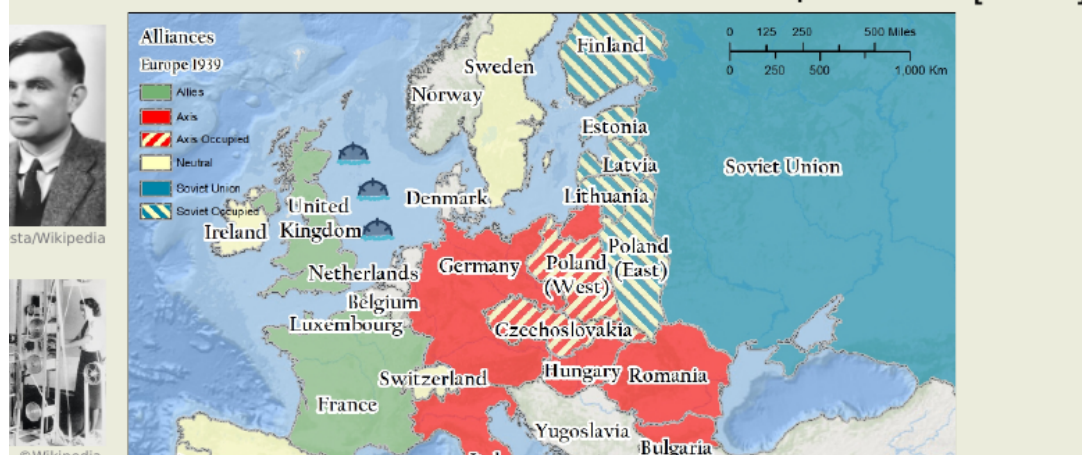


Figure 10: Historical / real-world CPA illustration included in the lecture slides.

Topic 4: PRF-based randomized SKE The lecture returns to the PRF-based encryption scheme from Lecture 09 and asks whether it is IND-CPA-secure. The key point is that the earlier deterministic-style comparison attack no longer works because encryption is randomized. The slide states the theorem:

Theorem / Key Claim

If F is a PRF, then the randomized PRF-based SKE construction is IND-CPA-secure.

The standard form of this construction (written explicitly here as an expanded learning aid) is:

$$\text{Gen}(1^n) : k \leftarrow \{0, 1\}^n,$$

$$\text{Enc}_k(m) : r \leftarrow \{0, 1\}^n, \quad c = (r, F_k(r) \oplus m),$$

$$\text{Dec}_k(r, y) : m = F_k(r) \oplus y.$$

Added by me: the slide image from Lecture 10 visibly uses fresh randomness together with a PRF output and XOR; the explicit equations above make that construction easier to study. They are the standard form consistent with the slide, but the full Lecture 09 text itself was not supplied in the current chat.

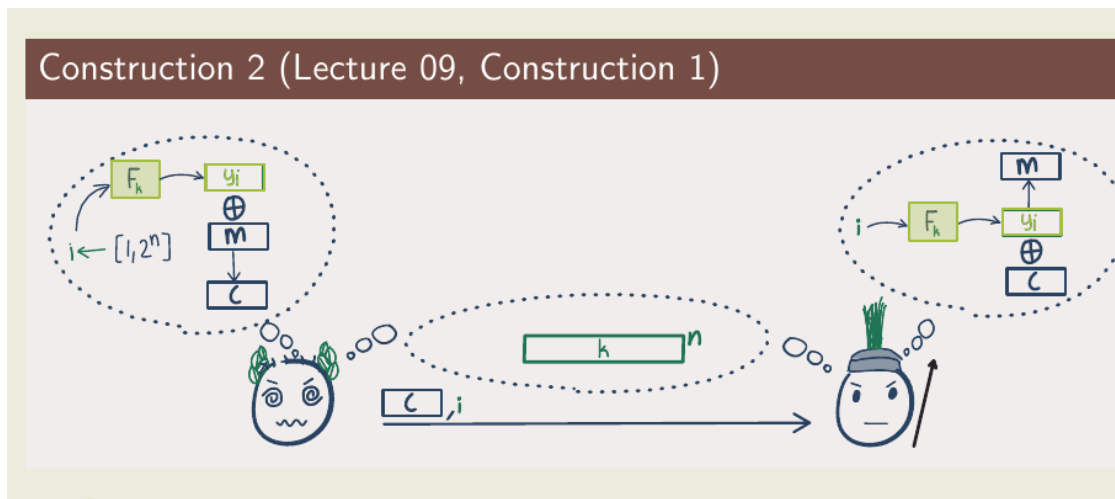


Figure 11: Lecture figure for the randomized PRF-based SKE construction.

Correctness of the PRF-based SKE – Added by me Correctness follows directly from XOR cancelation. During encryption,

$$y = F_k(r) \oplus m.$$

Decryption computes

$$F_k(r) \oplus y = F_k(r) \oplus (F_k(r) \oplus m) = m.$$

Thus the same key and the transmitted random value r are enough to recover the message.

Why the PRF-based SKE is IND-CPA-secure – proof intuition The proof is naturally organized around a hybrid argument, just as the lecture hints earlier. Consider two worlds:

Real world.

The encryption oracle uses the PRF $F_k(r)$.

Random-function world.

Replace F_k with a truly random function R . A PRF security argument says an efficient adversary should not be able to notice this replacement except with negligible probability.

In the random-function world, each fresh r gives a fresh random pad $R(r)$. Consequently the challenge ciphertext has the form

$$(r^*, R(r^*) \oplus m_b).$$

If r^* is fresh and is not reused, the mask $R(r^*)$ is uniformly random and independent of the challenge bit b . Therefore the challenge component is distributed identically for m_0 and m_1 , so the adversary's best strategy is essentially guessing.

There is one technical corner case: the challenge randomness r^* could collide with a previous encryption query's randomness. If r is n bits and there are only polynomially many queries, the

collision probability is negligible for sufficiently large n by the birthday bound. The standard proof handles this event explicitly.

Added by me – extra learning material

The proof in one sentence: PRF security lets us pretend the pad generator is a random function, fresh randomness makes the challenge input fresh with overwhelming probability, and a fresh random pad hides whether m_0 or m_1 was encrypted.

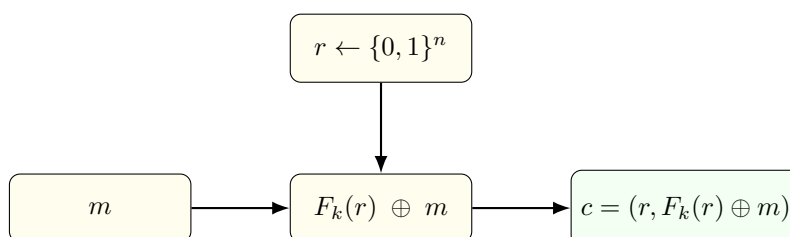
IND-CPA implies IND-EAV – and why the ordering matters The lecture asks for a hybrid proof that IND-CPA security implies the weaker eavesdropper notion IND-EAV. The intuition is simple: an eavesdropping adversary is just a special case of a CPA adversary that never uses its extra encryption-oracle power. Therefore any scheme secure against the stronger class of adversaries is automatically secure against the weaker class.

Added by me – extra learning material

How to think about the converse. The converse is not generally true: a security definition with fewer attacker capabilities need not protect against an attacker with strictly more capabilities. The deterministic-encryption discussion gives the right mental picture for why the stronger CPA definition demands additional structure (most importantly, randomized encryption).

What the lecture means by “stateless” A stateless encryption scheme does not need to remember a counter, previous ciphertext, or evolving internal state between encryptions. In the PRF-based construction, a fresh random value r is generated independently for each encryption. That random value is included in the ciphertext so that the receiver can recompute the same PRF output. This is precisely why the construction can support multiple messages without maintaining synchronized sender/receiver state.

PRF-based SKE: encryption/decryption picture – Added by me



Putting the whole lecture together The conceptual chain is:

$\boxed{\text{PRG}} \implies \boxed{\text{tree-based PRF}} \implies \boxed{\text{pseudorandom pads}} \implies \boxed{\text{randomized SKE}}$

with a separate threat-model chain

$\boxed{\text{IND-EAV}} \implies \boxed{\text{stronger attacker: CPA}} \implies \boxed{\text{need randomized encryption}}.$

In practice, block ciphers such as AES are the workhorse primitive used in place of a theoretical tree-based PRF construction. The lecture then previews the next step: viewing block ciphers as pseudorandom permutations and using modes of operation to handle long messages.

One-page memory sheet

Concept	What to remember
Length-doubling PRG	$G : \{0, 1\}^n \rightarrow \{0, 1\}^{2n}$; split as $G(s) = G_0(s) \ G_1(s)$.
Tree-based PRF	Start with $s_\varepsilon = k$; for $x = x_1 \cdots x_n$, set $s_i = G_{x_i}(s_{i-1})$; output s_n .
Efficiency	Full tree has 2^n leaves, but one PRF evaluation uses only n PRG evaluations.
PRF theorem	If G is a secure length-doubling PRG, the tree construction is a PRF (GGM-style hybrid proof).
Block cipher	Practical primitive; AES is the lecture's main example. Fixed AES key sizes: 128, 192, 256 bits.
CPA	Adversary can adaptively query an encryption oracle.
IND-CPA	Challenge ciphertext of m_0 versus m_1 should be indistinguishable: success only $1/2 + \text{negl}(n)$.
Deterministic encryption	Fails IND-CPA because equality of plaintexts can leak through repeated ciphertexts.
Randomized encryption	Fresh randomness prevents simple equality-comparison attacks.
PRF-based SKE	Standard form: $c = (r, F_k(r) \oplus m)$ with fresh r .
Why secure	Replace PRF by random function; fresh random mask hides the challenge bit except with negligible collision/proof error.
Next lecture	Block cipher as PRP; modes of operation for long messages.

Exam-style questions and answers

Exam / Revision Checklist

Q1. Given $G(s) = G_0(s) \| G_1(s)$, write $F_k(x_1 x_2 x_3)$. Answer:

$$F_k(x_1 x_2 x_3) = G_{x_3}(G_{x_2}(G_{x_1}(k))).$$

Q2. Why is the tree construction efficient? Answer: It never constructs the entire 2^n -leaf tree. One input identifies one path of length n , so only n PRG calls are needed.

Q3. State the PRF theorem. Answer: If G is a length-doubling PRG, the induced tree-based function family is a PRF.

Q4. What extra capability does a CPA adversary have? Answer: Access to an encryption oracle and the ability to adaptively choose plaintext queries.

Q5. Why can deterministic encryption not be IND-CPA-secure? Answer: The adversary can encrypt a known message itself and compare that ciphertext with the challenge; deterministic encryption preserves equality.

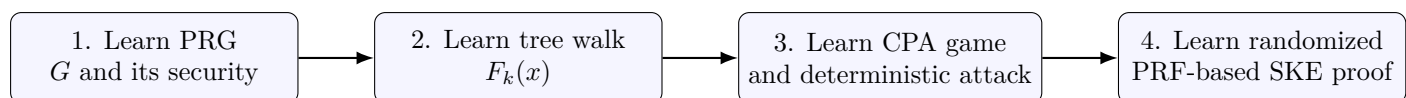
Q6. Why does $c = (r, F_k(r) \oplus m)$ hide m ? Answer: Replace F_k by a random function. For fresh r , $F_k(r)$ behaves like a fresh random pad, so XOR with it masks m .

Common mistakes to avoid

Common Pitfall

- Thinking that a PRF is literally a random function. It is only computationally indistinguishable from one to PPT adversaries.
- Saying the tree-based PRF takes exponential time. Constructing the entire table is exponential; evaluating one point is only linear in the input length.
- Forgetting that the PRF input r must be available during decryption. That is why r is included in the ciphertext.
- Writing a randomized encryption scheme but forgetting to sample fresh randomness for each encryption.
- Mixing up IND-EAV and IND-CPA. CPA gives the adversary strictly more power.
- Treating AES as a PRF on arbitrary-length messages. AES itself is a fixed-block primitive; a mode of operation specifies how it is used for longer data.

Further study from the lecture’s references The uploaded lecture cites GGM84 for the tree-based PRF, Goldreich’s *The Foundations of Cryptography, Volume 1* for the formal PRF proof, and Katz–Lindell’s *Introduction to Modern Cryptography* for the formal encryption construction. These references are the natural next step if a formal proof is required.



This study guide is a learning-oriented expansion of the uploaded Lecture 10 slide deck. Extra explanations are explicitly marked “Added by me”.